CMC Chat

Tài liệu ý tưởng và cách hoạt động
Bài tập lớn kỳ thực tập CMC - Tuần 6, 7, 8

Server chạy dưới dạng Windows service, lưu dữ liệu bằng SQLite, giao tiếp TCP.
Client viết bằng MFC, giao diện kiểu Messenger.

Mục lục

1. Tổng quan 3

2. Ý tưởng giao tiếp 4

3. Giao thức (Common\Protocol.h) 6

4. Server (Server\ChatServer.cpp) 8

5. Database (Server\Database.cpp) 10

6. Client (Client\) 12

7. Các luồng hoạt động chi tiết 14

8. Bảo mật và độ tin cậy 15

9. Cách build, chạy và demo 16

10. Bản đồ file 16

11. Hạn chế và hướng phát triển 17

1. Tổng quan

CMC Chat là ứng dụng nhắn tin 1-1 gồm 2 chương trình: **Server** (giữ dữ liệu, chuyển tin) và **Client** (giao diện người dùng). Bài đáp ứng các yêu cầu của 3 tuần cuối kỳ thực tập:

Tuần	Yêu cầu	Đáp ứng bằng
6	Ý tưởng giao tiếp giữa client với client	Mô hình client - server qua TCP, gói tin header + JSON (mục 2, 3)
6	DB lưu user, tin nhắn	SQLite, 2 bảng users / messages (mục 5)
6	Đăng ký, đăng nhập	Action REGISTER / LOGIN, mật khẩu băm SHA-256 + PBKDF2 (mục 5.3, 7)
6	Chạy dưới dạng Windows service	ChatServer.exe install, dừng không treo (mục 4.4)
7	Client đủ giao diện, kết nối, đăng ký, đăng nhập	3 màn hình: Đăng nhập, Tạo tài khoản, Chat (mục 6)
8	Ghép Client - Server, gửi được tin cho nhau	Tin realtime, tin offline, lịch sử, đã xem, đang nhập... (mục 7)

1.1 Thành phần

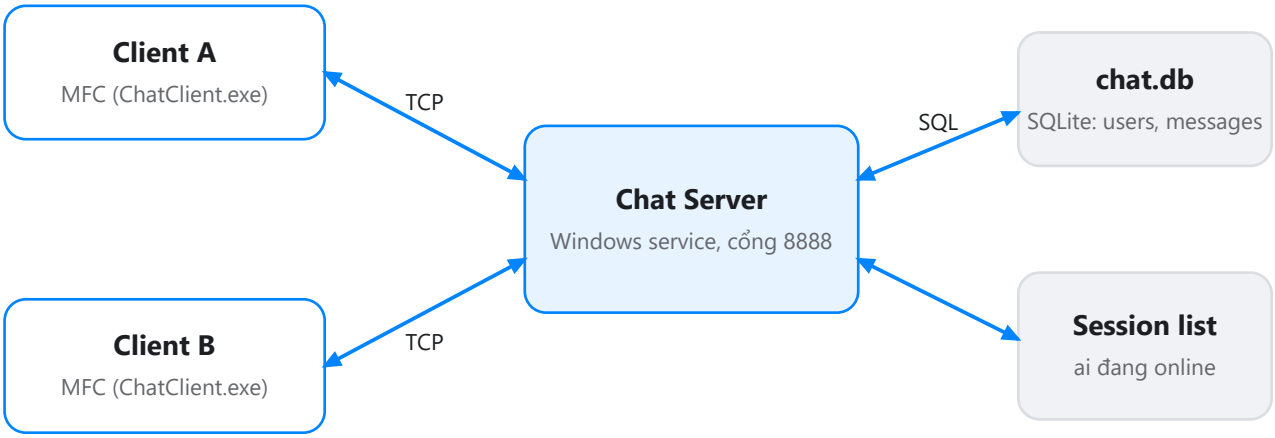
Thư mục	Nội dung
Common\	Protocol.h (giao thức), Json.h (thư viện JSON nhỏ) - dùng chung cho cả Client và Server
Server\	ChatServer.cpp (mạng + Windows service), Database.cpp/.h (SQLite)
Client\	Ứng dụng MFC: màn hình đăng nhập, đăng ký, chat; lớp kết nối mạng; các control tự vẽ
ChatApp.slnx	Solution mở cả 2 project trong Visual Studio

1.2 Công nghệ

- **C++ / MFC** cho Client, **C++ thuần + WinAPI** cho Server.
- **Winsock (TCP)** để truyền dữ liệu.
- **SQLite** có sẵn trong Windows (winsqlite3.dll) - không cần cài thêm gì.
- **BCrypt API** của Windows để băm mật khẩu (SHA-256, PBKDF2) và sinh salt ngẫu nhiên.
- **GDI+** để vẽ giao diện mượt (hình tròn, bo góc, ảnh avatar).

2. Ý tưởng giao tiếp

Hai client **không nói chuyện trực tiếp** với nhau mà luôn đi qua Server. Mỗi client mở **1 kết nối TCP** tới Server và giữ kết nối đó trong suốt thời gian đăng nhập.



Mọi tin nhắn đều đi qua Server: Client không nói chuyện trực tiếp với nhau

Hình 1. Kiến trúc tổng thể

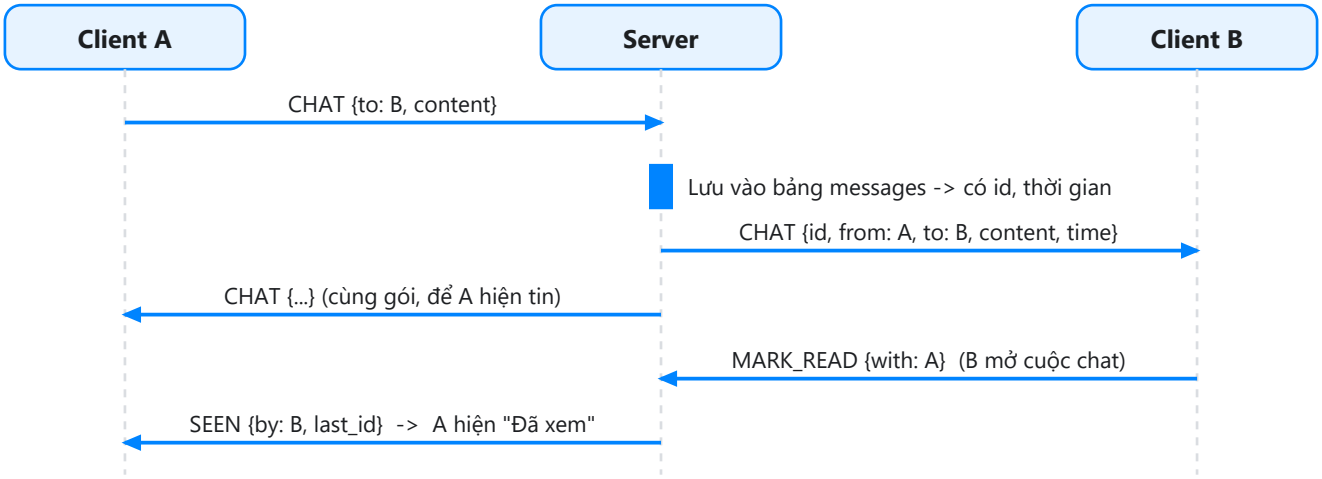
2.1 Vì sao đi qua Server?

- Server biết ai đang online, nên chuyển tin **ngay lập tức** tới người nhận.
- Người nhận đang offline thì Server **lưu tin vào database**, lần sau đăng nhập vẫn đọc được.
- Client không cần biết IP của nhau - chỉ cần biết IP của Server.
- Mọi kiểm tra (đăng nhập đúng chưa, người nhận có tồn tại không...) làm tập trung ở Server.

2.2 Vì sao chọn TCP?

Chat cần Server **chủ động đẩy** tin tới client ngay khi có tin mới. TCP giữ kết nối liên tục nên làm được việc này đơn giản và nhẹ. Nếu dùng HTTP / REST API thì client phải hỏi Server liên tục (polling) mới biết có tin mới. WebSocket cũng giữ kết nối nhưng phải tự cài thêm 1 lớp giao thức trong C++. TCP còn đảm bảo dữ liệu tới **đủ và đúng thứ tự**.

2.3 Gửi tin A → B diễn ra thế nào?



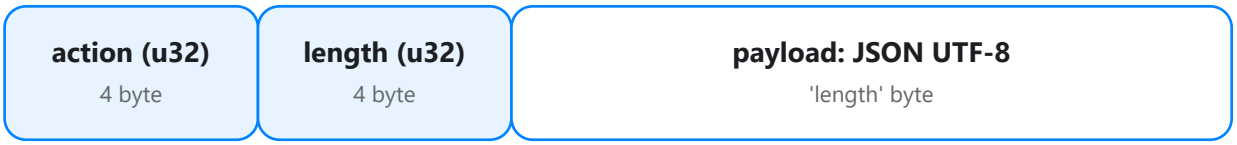
Hình 2. Luồng gửi một tin nhắn

Điểm quan trọng: A chỉ hiện tin của mình khi nhận lại gói CHAT từ Server (không tự hiện ngay khi bấm Gửi). Nhờ vậy tin trên 2 máy luôn có **cùng id, cùng thứ tự**, và nếu Server không lưu được thì A biết ngay.

3. Giao thức (Common\Protocol.h)

3.1 Định dạng gói tin

TCP là một **dòng byte liên tục**, không tự chia thành từng tin. Vì vậy mỗi gói tin phải có độ dài ở đầu để bên nhận biết gói kết thúc ở đâu:



Header 8 byte (big-endian): 4 byte đầu cho biết loại lệnh, 4 byte sau cho biết cần đọc thêm bao nhiêu byte

Hình 3. Cấu trúc 1 gói tin

Bên nhận làm theo thứ tự: đọc đủ 8 byte header → lấy **action** (loại lệnh) và **length** → đọc tiếp đúng **length** byte → phân tích JSON. Nội dung là JSON UTF-8 nên tiếng Việt, emoji, dấu ngoặc kép, xuống dòng... đều truyền đúng. Ví dụ gói đăng nhập:

```
Header : 00 00 00 02 | 00 00 00 5A        (action = 2 = LOGIN, length = 90 byte)
Payload: {"username":"ngomanhhieu","password":"a4ayc/80/OGda4BO/1o/V0etpOqiLx1JwB5S3beHW0s="}
```

Mọi phản hồi của Server đều có trường **status** là **"ok"** hoặc **"failed"** (kèm **reason** là lý do lỗi tiếng Việt).

3.2 Danh sách action

Mã	Action	Client → Server	Server → Client
1	REGISTER	{username, password, display_name}	{status, reason?}
2	LOGIN	{username, password}	{status, username, display_name, has_avatar}
3	LOGOUT	{}	(Server đóng kết nối)
4	GET_USERS	{}	{users: [{username, display_name, online, unread, last_from, last_content, last_time, has_avatar, last_seen}]}
5	GET_MESSAGES	{with}	{with, messages: [{id, from, to, content, time}], seen_id}
6	CHAT	{to, content}	{id, from, to, content, time} - gửi cho cả 2 bên
7	MARK_READ	{with}	(không trả lời; Server gửi SEEN cho người kia)
8	USER_STATUS	-	{username, display_name, online, last_seen} - ai đó online/offline/vừa đăng ký
9	GET_AVATAR	{username}	{username, data: ảnh base64}
10	SET_AVATAR	{data}	{status}; mọi người nhận GET_AVATAR mới
11	TYPING	{to}	{from} - "đang nhập..."
12	SEEN	-	{by, last_id} - người kia đã xem tới tin last_id
99	ERROR_MSG	-	{reason}

3.3 Các hàm chính trong Protocol.h

Hàm	Việc làm
Build(action, json)	Ghép header 8 byte + JSON thành chuỗi byte để gửi
SendAll(sock, data)	Gửi đủ toàn bộ dữ liệu (send() có thể chỉ gửi được một phần nên phải lặp)
RecvAll(sock, buf, len, stop)	Đọc đủ len byte. Cứ 200 ms kiểm tra cờ stop 1 lần nên thread không bao giờ bị kẹt
RecvPacket(sock, pkt, stop)	Đọc 1 gói hoàn chỉnh: header → payload → JSON. Gói quá 4 MB thì từ chối
HashPasswordForWire(pw)	Client băm mật khẩu: base64(SHA-256(mật khẩu))
Base64Encode / Decode	Mã hoá dữ liệu nhị phân (hash, ảnh) để đặt được vào JSON

4. Server (Server\ChatServer.cpp)

4.1 Các thread

Thread	Nhiệm vụ
Thread chính	Khởi động server, rồi đứng chờ **g_stopEvent** (lệnh dừng). Có lệnh thì gọi StopServer()
AcceptLoop (1 thread)	Gọi accept() để nhận client mới. Mỗi client mới → tạo 1 Session + 1 ClientThread
ClientThread (mỗi client 1 thread)	Vòng lặp: RecvPacket → HandlePacket → lặp lại, cho tới khi client ngắt kết nối

4.2 Session - quản lý ai đang online

Mỗi kết nối là 1 **Session** gồm: socket, địa chỉ IP, user đã đăng nhập (id = 0 nếu chưa), và 1 khoá gửi (**sendMtx**). Tất cả session nằm trong **g_sessions**, có khoá **g_sessMtx** vì nhiều thread cùng đọc/ghi.

- **SendTo(session, action, json)**: gửi 1 gói tới 1 client. Khoá sendMtx để 2 thread không gửi xen kẽ vào cùng 1 socket.
- **SessionsOf(userId)**: lấy các kết nối của 1 user (1 người có thể đăng nhập ở nhiều cửa sổ).
- **IsOnline(userId)**: user có ít nhất 1 kết nối là online.
- **BroadcastStatus(user, online)**: báo cho mọi người biết ai đó vừa online/offline (USER_STATUS).

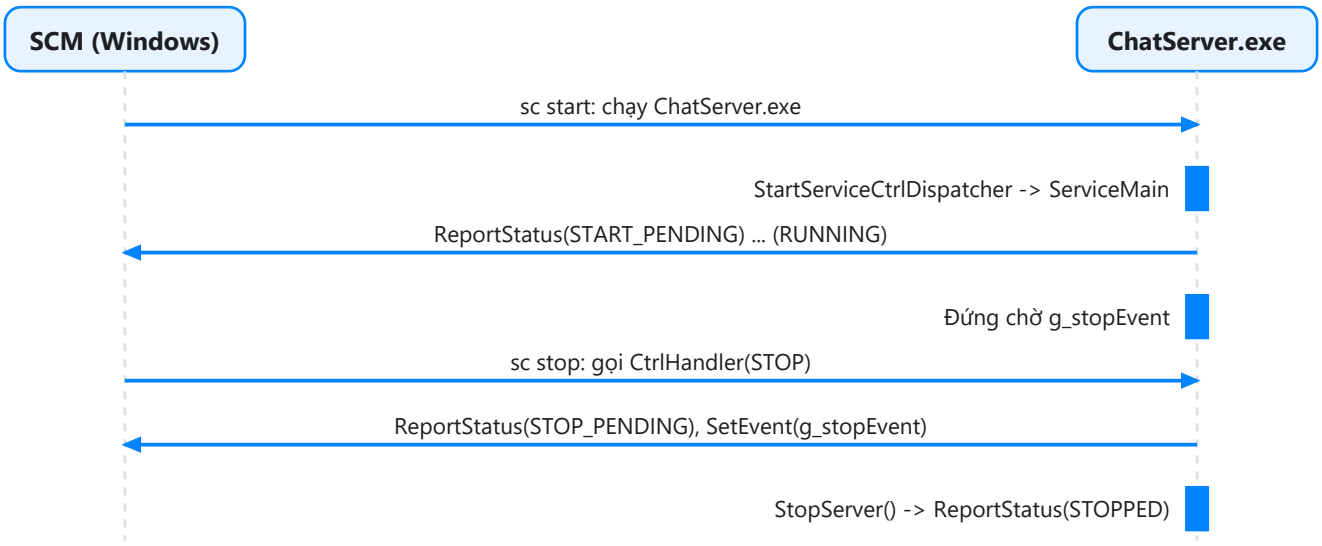
4.3 Xử lý từng action

HandlePacket nhìn vào mã action rồi gọi hàm tương ứng. REGISTER, LOGIN, LOGOUT không cần đăng nhập; các action còn lại thì session phải đăng nhập rồi, nếu không Server trả ERROR "Bạn chưa đăng nhập".

Hàm	Làm gì
OnRegister	Kiểm tra dữ liệu, lưu user mới vào DB, báo cho mọi người có user mới
OnLogin	So mật khẩu với DB; đúng thì gắn user vào session, báo mọi người "đang online"
OnGetUsers	Lấy danh sách user + số tin chưa đọc + tin cuối + online/offline
OnGetMessages	Lấy 300 tin gần nhất giữa 2 người + seen_id (tin mới nhất người kia đã xem)
OnChat	Lưu tin vào DB (giờ UTC), gửi CHAT cho mọi kết nối của người nhận và người gửi
OnMarkRead	Đánh dấu đã đọc, rồi gửi SEEN cho người gửi để hiện "Đã xem"
OnTyping	Chuyển tín hiệu "đang nhập..." tới người nhận (không lưu DB)
OnGetAvatar / OnSetAvatar	Đọc / lưu ảnh avatar (chỉ nhận PNG/JPG ≤ 512 KB), báo mọi người ảnh mới

4.4 Chạy dưới dạng Windows service

Windows service là chương trình chạy nền do **SCM** (Service Control Manager) của Windows quản lý: không có cửa sổ, chạy được cả khi chưa ai đăng nhập máy, bật/tắt bằng services.msc hoặc lệnh sc.



Hình 4. Vòng đời của service

4.5 Dừng service mà không bị treo

- **CtrlHandler** chỉ bật g_stopEvent rồi trả về ngay; việc dọn dẹp làm ở thread chính.
- Đóng socket lắng nghe → accept() trả về → AcceptLoop thoát.
- Mỗi ClientThread đọc dữ liệu bằng RecvAll: cứ 200 ms kiểm tra cờ dừng, nên thấy lệnh dừng gần như ngay lập tức (trên Windows, gọi shutdown() từ thread khác không làm recv() đang chờ trả về, nên phải dùng select có timeout).
- send() có timeout 3 giây; Server chờ các client thread thoát tối đa 5 giây.
- Kết quả test: dừng service khi còn 2 client đang kết nối mất khoảng **0,1 - 0,2 giây**.

4.6 Chế độ console (bấm F5 trong Visual Studio)

Khi chạy ChatServer.exe không qua SCM (vd bấm F5), StartServiceCtrlDispatcher báo lỗi, chương trình tự chuyển sang chạy trong cửa sổ console để dễ debug. Gõ **users** để xem tài khoản trong DB, gõ **q** để dừng. Lệnh dừng đi qua đúng đường dừng của service.

4.7 Thư mục dữ liệu

Database và log nằm cố định ở **C:\ProgramData\CMCChat** (chat.db, ChatServer.log), để dù chạy F5 hay service, bản x86 hay x64 thì cũng dùng chung 1 database. Thư mục được tạo với quyền cho cả tài khoản SYSTEM (service) và tài khoản thường (F5) cùng đọc/ghi.

5. Database (Server\Database.cpp)

SQLite là database **nhúng**: không có chương trình database chạy riêng, toàn bộ dữ liệu nằm trong 1 file. Server mở file khi khởi động. Các câu SQL được viết trong code (Database.cpp); bảng được tạo tự động bằng CREATE TABLE IF NOT EXISTS, nên sang máy mới chỉ cần chạy server là có đủ bảng.

5.1 Cấu trúc 2 bảng

Bảng users	Kiểu	Ý nghĩa
id	INTEGER	Khoá chính, tự tăng
username	TEXT	Tên đăng nhập, duy nhất, không phân biệt hoa thường
display_name	TEXT	Tên hiển thị (có dấu tiếng Việt)
password_hash	BLOB	PBKDF2(mật khẩu đã băm, salt) - 32 byte
salt	BLOB	16 byte ngẫu nhiên, mỗi user 1 salt riêng
created_at	INTEGER	Thời điểm đăng ký (UTC timestamp)
avatar	BLOB	Ảnh avatar PNG/JPG (có thể rỗng)
last_seen	INTEGER	Lần cuối online (để hiện "Hoạt động X phút trước")

Bảng messages	Kiểu	Ý nghĩa
id	INTEGER	Khoá chính, tự tăng - cũng là thứ tự tin
sender_id	INTEGER	Khoá ngoại → users.id (người gửi)
receiver_id	INTEGER	Khoá ngoại → users.id (người nhận)
content	TEXT	Nội dung UTF-8
created_at	INTEGER	Thời điểm gửi (UTC timestamp)
is_read	INTEGER	0 = chưa đọc, 1 = đã đọc

Có thêm chỉ mục **(sender_id, receiver_id, id)** để lấy lịch sử chat giữa 2 người nhanh. DB cũ thiếu cột mới (avatar, last_seen) thì Server tự thêm bằng ALTER TABLE khi khởi động - không mất dữ liệu.

5.2 Chống SQL injection: prepared statement

Không bao giờ ghép dữ liệu người dùng vào chuỗi SQL. Câu SQL dùng dấu **?**, giá trị truyền riêng qua Bind():

```
Stmt st(m_db, "SELECT id, username, display_name FROM users WHERE username = ?");
st.Bind(1, username);      // dù người dùng nhập  hieu' --  thì vẫn chỉ là 1 chuỗi bình thường
```

5.3 Bảo mật mật khẩu

Bước	Ở đâu	Làm gì
1	Client	Băm SHA-256 mật khẩu rồi gửi dạng base64 - mật khẩu thật không đi qua mạng
2	Server (đăng ký)	Sinh salt 16 byte ngẫu nhiên, tính PBKDF2-SHA256(hash, salt, 100 000 vòng), lưu hash + salt

Bước	Ở đâu	Làm gì
3	Server (đăng nhập)	Tính lại PBKDF2 với salt đã lưu, so sánh từng byte (không dừng sớm)

PBKDF2 cố ý chậm (vài chục ms mỗi lần) nên dù database bị lộ thì cũng rất khó dò ngược ra mật khẩu. Salt riêng cho mỗi user làm 2 người cùng mật khẩu vẫn có hash khác nhau.

6. Client (Client\)

6.1 Vòng đời ứng dụng (ChatClient.cpp)

App chạy vòng lặp: **Màn hình đăng nhập → Màn hình chat**. Đăng xuất hoặc mất kết nối thì quay lại màn hình đăng nhập; đóng cửa sổ thì thoát app. Một đối tượng **CNetClient** giữ kết nối TCP, dùng chung cho cả 2 màn hình.

6.2 Nhận dữ liệu không làm đơ giao diện (NetClient.cpp)

Giao diện MFC chạy trên 1 thread (UI thread). Nếu đọc socket trên thread này thì app sẽ đơ khi chờ dữ liệu. Vì vậy:

- 1 **thread nền** chỉ làm việc đọc gói tin (RecvPacket) rồi bỏ vào **hàng đợi**.
- Thread nền gửi message **WM_NET_EVENT** cho cửa sổ đang hiện.
- Cửa sổ nhận WM_NET_EVENT thì gọi **Pop()** lấy gói ra xử lý trên UI thread (an toàn khi vẽ giao diện).
- Dùng hàng đợi để gói tin tới đúng lúc chuyển từ màn hình đăng nhập sang màn hình chat không bị mất.
- Mất kết nối → hàng đợi nhận gói đặc biệt NET_CLOSED → app báo lỗi và quay về đăng nhập.

6.3 Các màn hình và lớp

Lớp / File	Vai trò
CLoginDlg	Nhập server, cổng, tên đăng nhập, mật khẩu. Nhớ server và tên đăng nhập lần trước (registry)
CRegisterDlg	Tạo tài khoản; kiểm tra dữ liệu ngay ở client (3-20 ký tự, mật khẩu ≥ 6, nhập lại khớp)
CMainDlg	Màn hình chat: danh sách người dùng, tìm kiếm, khung chat, ô nhập, xử lý mọi gói tin từ Server
CContactList	Danh sách người dùng tự vẽ: avatar, chấm online, tin cuối, giờ, số tin chưa đọc, "Đang nhập..."
CChatView	Khung tin nhắn tự vẽ: bong bóng, gom nhóm tin, vạch thời gian, cuộn, chuột phải để sao chép
CComposerEdit	Ô nhập tin dùng RichEdit, tự cao lên theo số dòng, hỗ trợ bộ gõ tiếng Việt
CThemedDialog	Lớp nền: ô nhập bo góc, nút xanh, nút link, chống nhấp nháy
theme (Theme.cpp)	Màu sắc, font, DPI, định dạng thời gian, vẽ avatar, đọc/thu nhỏ ảnh

6.4 Giao diện được vẽ như thế nào

- **Tự vẽ (owner-draw)**: danh sách, khung chat, nút bấm đều tự vẽ bằng GDI/GDI+ thay vì dùng giao diện mặc định của Windows.
- **Vẽ vào bitmap tạm rồi copy 1 lần** (double buffering) để không nhấp nháy.
- **GDI+** vẽ hình tròn, bo góc có khử răng cưa; avatar ảnh được tô vào hình tròn bằng TextureBrush.
- **DPI**: mọi kích thước viết ở 96 DPI rồi nhân hệ số qua theme::S(), nên chữ sắc nét trên màn hình scale 125%/150%.

6.5 Khung tin nhắn (CChatView)

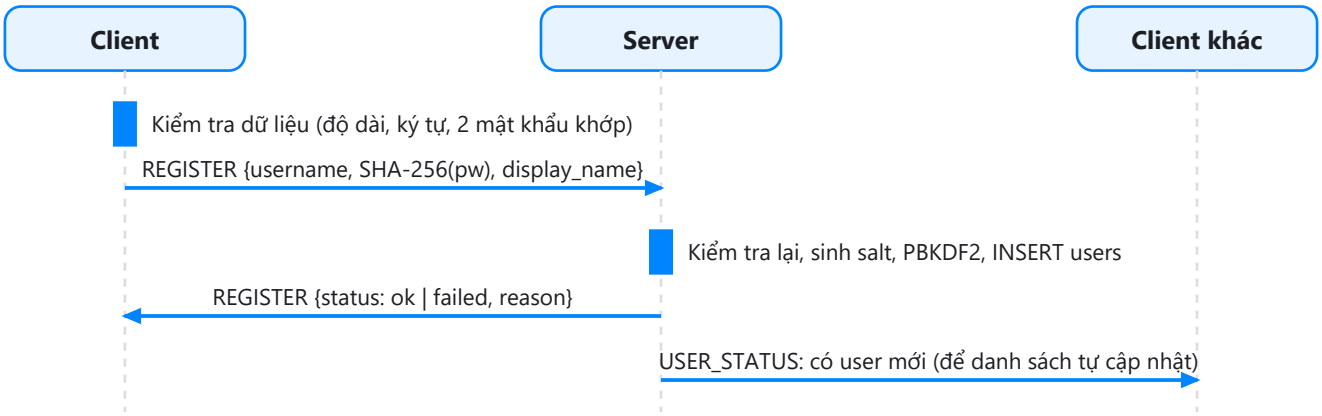
- **Layout()** tính trước vị trí mọi bong bóng (đo chữ bằng DrawText CALCRECT), **OnPaint()** chỉ vẽ phần đang thấy.
- Tin liên tiếp của cùng 1 người cách nhau < 2 phút được gom thành 1 nhóm (avatar và giờ chỉ hiện ở tin cuối nhóm).
- Cách nhau ≥ 15 phút hoặc khác ngày thì chèn vạch thời gian ("14:05", "14:05, Hôm qua"...).
- Tin ít thì đẩy xuống sát đáy giống Messenger; có tin mới mà đang ở cuối thì tự cuộn xuống.

6.6 Ô nhập tin và bộ gõ tiếng Việt

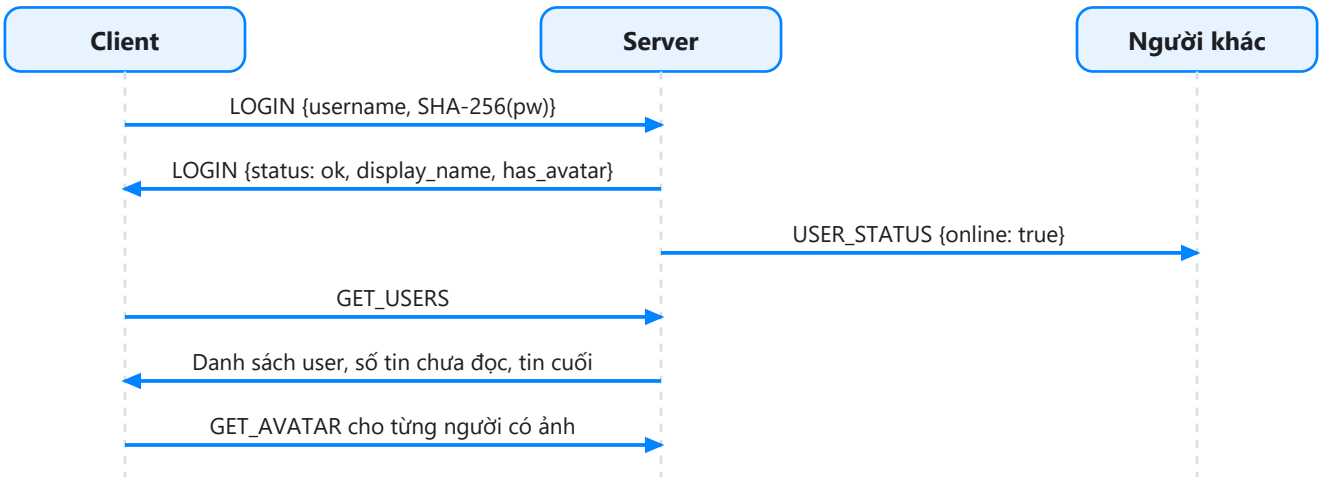
Ô nhập dùng **RichEdit** thay cho EDIT thường. Với bộ gõ IME (vd Telex của Windows), chữ đang soạn chưa nằm trong ô EDIT nên EDIT không tự xuống dòng. RichEdit giữ chữ đang soạn ngay trong văn bản. Bộ gõ Telex của Windows chạy qua **TSF** (Text Services Framework) nên phải bật TSF cho RichEdit (EM_SETEDITSTYLE với cờ SES_USECTF) và khởi tạo COM khi app chạy, nếu không gõ ra chữ không dấu. Sau mỗi lần nhấn phím, ô nhập tính lại số dòng và tự cao lên (tối đa 6 dòng, quá thì hiện thanh cuộn). **Enter** = gửi, **Shift+Enter** = xuống dòng.

7. Các luồng hoạt động chi tiết

7.1 Đăng ký



7.2 Đăng nhập và mở màn hình chat



7.3 Mở 1 cuộc trò chuyện

Bấm vào 1 người trong danh sách → gửi **GET_MESSAGES** (lần đầu) để lấy 300 tin gần nhất, và **MARK_READ** nếu có tin chưa đọc. Server đánh dấu đã đọc rồi gửi **SEEN** cho người kia → bên kia hiện "Đã xem" dưới tin cuối.

7.4 Tin nhắn offline

Người nhận offline thì Server vẫn lưu tin (is_read = 0). Khi họ đăng nhập, GET_USERS trả về số tin chưa đọc → danh sách hiện số màu xanh. Mở cuộc chat thì GET_MESSAGES lấy đủ tin.

7.5 "Đang nhập..."

Khi gõ phím trong ô nhập, client gửi **TYPING {to}** (tối đa 1 lần / 2 giây). Server chuyển sang người nhận; bên đó hiện "Đang nhập..." ở tiêu đề, ở danh sách và 1 bong bóng 3 chấm trong khung chat. Không nhận thêm tín hiệu trong 4 giây, hoặc tin nhắn tới, thì tự tắt. Tín hiệu này không lưu vào DB.

7.6 Online / offline và "Hoạt động X phút trước"

Đăng nhập → Server gửi USER_STATUS online cho mọi người. Ngắt kết nối cuối cùng → Server lưu **last_seen** rồi gửi USER_STATUS offline. Client hiện "Đang hoạt động" (chấm xanh) hoặc "Hoạt động 5 phút trước" và tự cập nhật mỗi giây.

7.7 Đổi avatar

Bấm vào avatar của mình (góc trên trái) → chọn ảnh → client cắt vuông (ảnh dọc lấy phần trên để giữ khuôn mặt), thu về 160×160, nén PNG, gửi **SET_AVATAR**. Server kiểm tra (PNG/JPG, ≤ 512 KB), lưu vào DB và gửi ảnh mới cho mọi người đang online.

7.8 Mất kết nối

Server tắt hoặc mạng đứt → thread nền của client đọc thất bại → gói NET_CLOSED → app báo "Mất kết nối tới server" và quay về màn hình đăng nhập. Phía Server, client đột ngột biến mất thì recv() lỗi (hoặc keep-alive phát hiện), ClientThread dọn dẹp và báo mọi người người đó offline.

8. Bảo mật và độ tin cậy

Vấn đề	Cách xử lý
Lộ mật khẩu trên mạng	Chỉ gửi SHA-256 của mật khẩu
Lộ database	PBKDF2 + salt riêng, 100 000 vòng
SQL injection	Prepared statement cho mọi câu SQL
Gói tin bất thường	Giới hạn 4 MB/gói, 8 000 byte/tin, 512 KB/ảnh; JSON sai thì báo lỗi, không crash
Nhiều thread cùng dùng dữ liệu	Mutex cho danh sách session, cho DB và cho từng socket khi gửi
Client treo không đọc	send() timeout 3 giây, không kéo Server treo theo
Dừng service bị treo	select 200 ms + chờ tối đa 5 giây
Tin trùng / sai thứ tự	Mỗi tin có id do Server cấp; client bỏ qua tin đã có

9. Cách build, chạy và demo

9.1 Build

Mở **Week 6\MFC_CMC_Week6_ChatApp\ChatApp.slnx** trong Visual Studio, chọn Debug | x64, Build Solution. File chạy: Server\x64\Debug\ChatServer.exe và Client\x64\Debug\ChatClient.exe.

9.2 Chạy server

Cách	Lệnh
Debug (console)	Bấm F5 project ChatServer. Gõ users để xem tài khoản, q để dừng
Cài service	cmd (Admin): ChatServer.exe install
Chạy / dừng service	sc start CMCChatServer / sc stop CMCChatServer (hoặc services.msc)
Gỡ service	ChatServer.exe uninstall

Service và F5 dùng chung cổng 8888 nên không chạy cùng lúc được. Muốn build lại server khi service đang chạy thì phải Stop service trước (file exe đang bị khoá).

9.3 Chạy client

Mở ChatClient.exe (có thể mở nhiều cửa sổ để giả làm nhiều người). Nhập IP server (127.0.0.1 nếu cùng máy), cổng 8888. Chưa có tài khoản thì bấm "Đăng ký ngay". Muốn chạy trên máy khác: mở firewall cổng 8888 trên máy server và nhập IP LAN của máy server.

9.4 Kịch bản demo gợi ý

- Trình bày ý tưởng: Hình 1, Hình 2, Hình 3 trong tài liệu này.
- Mở services.msc cho thấy CMC Chat Server đang Running.
- Đăng ký tài khoản mới (thử lỗi: tên quá ngắn, mật khẩu không khớp, trùng tên), đăng nhập (thử sai mật khẩu).
- Mở 2 client, nhắn qua lại: tin realtime, "Đang nhập...", "Đã xem", tiếng Việt, emoji, tin dài tự xuống dòng.
- Đăng xuất 1 người → bên kia hiện "Hoạt động vừa xong"; gửi tin offline → đăng nhập lại thấy số chưa đọc.
- Đổi avatar bằng cách bấm vào avatar của mình.
- Stop service khi 2 client đang mở → dừng ngay, client báo mất kết nối; Start lại → lịch sử vẫn còn.

10. Bản đồ file

File	Nội dung chính
Common\Protocol.h	Mã action, định dạng gói, gửi/nhận gói, base64, băm mật khẩu phía client
Common\Json.h	Tạo / phân tích JSON (hỗ trợ UTF-8, emoji)
Server\ChatServer.cpp	Session, xử lý action, AcceptLoop, ClientThread, Windows service, console, install/uninstall
Server\Database.cpp/.h	Tạo bảng, đăng ký, đăng nhập, danh sách user, lưu/lấy tin, đã đọc, avatar, last_seen

File	Nội dung chính
Client\ChatClient.cpp	Điểm vào app, vòng lặp đăng nhập → chat
Client\NetClient.cpp/.h	Kết nối TCP, thread nhận, hàng đợi gói tin
Client\LoginDlg, RegisterDlg	Màn hình đăng nhập, tạo tài khoản
Client\MainDlg.cpp/.h	Màn hình chat, xử lý gói tin từ Server
Client\ChatView, ContactList	Khung tin nhắn, danh sách người dùng (tự vẽ)
Client\ComposerEdit	Ô nhập tin RichEdit
Client\ThemedDialog, Theme	Giao diện chung, màu, font, DPI, avatar

11. Hạn chế và hướng phát triển

- Dữ liệu trên mạng chưa mã hoá: có thể bọc TCP bằng TLS (Schannel của Windows).
- Chưa có chat nhóm, gửi file/ảnh trong tin nhắn.
- Client chưa tự kết nối lại khi mạng chập chờn (hiện phải đăng nhập lại).
- Chỉ tải 300 tin gần nhất; có thể thêm "cuộn lên để tải thêm".
- Bản Debug cần thư viện của Visual Studio; muốn gửi client sang máy khác nên build Release với MFC tĩnh.